

NOCTURNE BLOOD RUN

PROJE RAPORU

Unity Scripting-Visualization Basics Uygulama Analizi

Öğrenci	Ayşenisa Yaşar
Numara	222511022
Tarih	19.05.2026

Bu rapor, Nocturne Blood Run projesinde kullanılan temel Unity programlama ve görselleştirme kavramlarını; oyun sahnesi, script mimarisi ve kod referansları üzerinden ayrıntılı biçimde açıklamak için hazırlanmıştır.



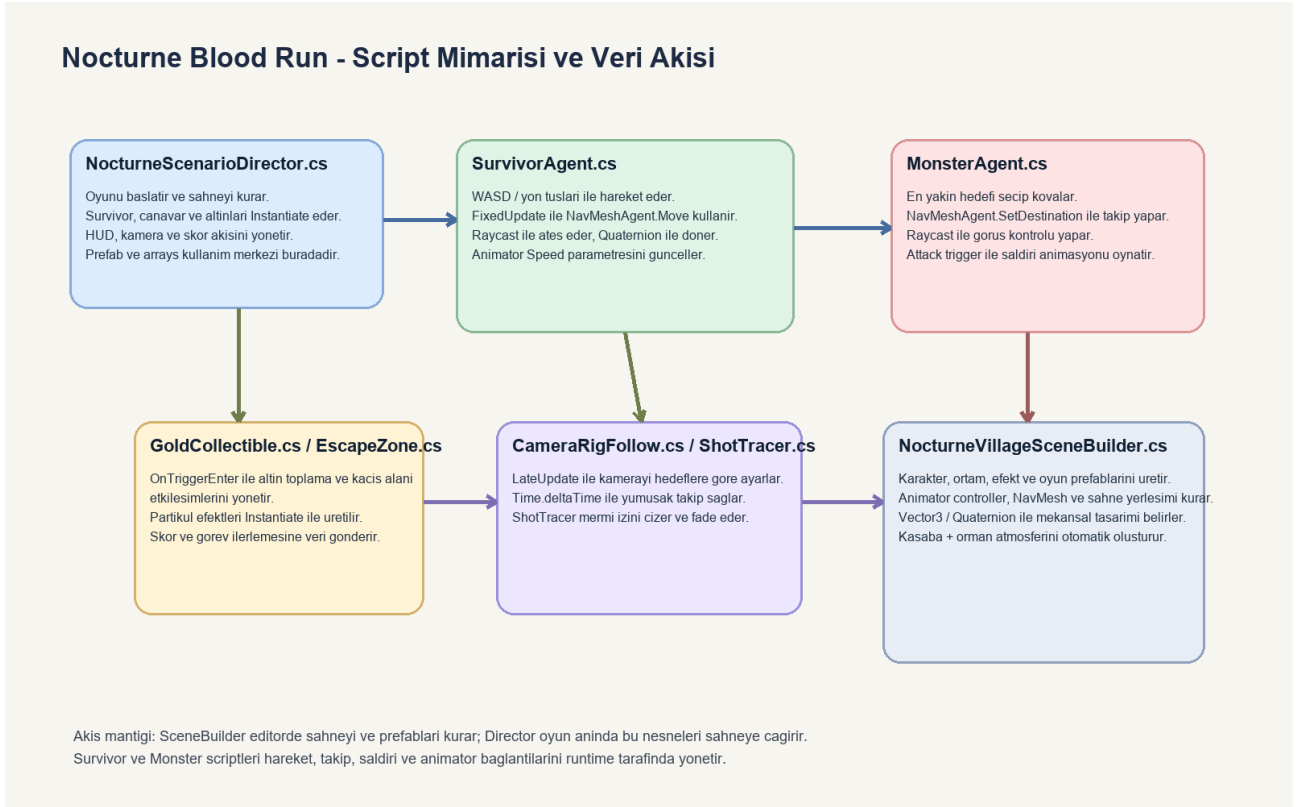
Görsel 1. Kasaba, orman, kaçış yolları, başlangıç noktaları ve altın yerleşimlerinin üstten şematik gösterimi.

1. Proje Özeti

Nocturne Blood Run, karanlık bir sonbahar kasabasında geçen kaçış ve hayatta kalma temalı bir Unity projesidir. Oyunda Remy ve Peasant Girl karakterleri kasabadan ve ormandan kaçır, altın toplar, belirli skora ulaştıktan sonra silahlarını açar ve onları avlamaya çalışan canavarı durdurmaya çalışır.

- İki oyuncu karakter farklı kontrol şemaları ile yönetilir: Remy için WASD, Peasant Girl için yön tuşları kullanılır.
- Canavar, Navigation Mesh üzerinde oyuncuları aktif biçimde izler ve görüş doğrulaması yaptıktan sonra saldırı başlatır.
- Altın toplama, silah kilidi açma, raycast ile ateş etme, partikül efektleri ve skor takibi tek akışta birleştirilmiştir.
- Kasaba ve orman ortamı editör tarafında otomatik olarak inşa edilir; böylece sahne kurulumu tekrar üretilebilir hale getirilmiştir.

2. Proje Görselleri ve Mimari Akış



Görsel 2. Scriptler arasındaki sorumluluk paylaşımı ve veri akışı.

Yukarıdaki mimari akışta editör tarafındaki SceneBuilder, oyunun sahne ve prefab üretimini üstlenirken runtime tarafında Director, SurvivorAgent ve MonsterAgent scriptleri oynanışı sürdürmektedir. Böylece proje hem sahne kurulumunda hem de oyun anında modüler bir yapıya kavuşmuştur.

3. Zorunlu Unity Kavramlarının Kullanım Tablosu

Kavram	Kullanıldığı Yer	Projede Görevi
--------	------------------	----------------

Prefab	NocturneVillageSceneBuilder.cs 338-381, 427-528; NocturneScenarioDirector.cs 199-238	Karakter, ev, ağaç, lamba, altın ve efekt nesnelerinin tekrar kullanılabilir şekilde üretilmesi.
Vector3 / Quaternion	SurvivorAgent.cs 100-126, 303-309; CameraRigFollow.cs 58-63; SceneBuilder.cs 602-643	Hareket yönü, dönüş, kamera bakışı, mermi çıkış noktası ve sahne yerleşimi.
Partikül Efektleri	SceneBuilder.cs 358-372, 1558-1703; GoldCollectible.cs 52-54; MonsterAgent.cs 220, 255	Doğuş, vurulma, altın toplama, sis, yaprak ve saldırı efektleri.
Instantiate	NocturneScenarioDirector.cs 199-238; SurvivorAgent.cs 309-339; EscapeZone.cs 35	Oyuncu, canavar, altın, tracer ve partikül nesnelerini çalışma anında üretme.
RayCast	SurvivorAgent.cs 312-326; MonsterAgent.cs 178-182	Ateşin hedefe çarpıp çarpmadığını ve canavarın oyuncuyu görüp görmediğini test etme.
FixedUpdate / LateUpdate / DeltaTime	SurvivorAgent.cs 93-126; CameraRigFollow.cs 19-63; GoldCollectible.cs 26-27	Fizik temelli hareket, kamera takibi ve zamana bağlı animasyon geçişleri.
Trigger / Collision	GoldCollectible.cs 17-21, 37-58; EscapeZone.cs 17-35	Altın toplama ve kaçış alanı gibi bölgesel etkileşimlerin tetiklenmesi.
Arrays	NocturneScenarioDirector.cs 10-27, 193-243; CameraRigFollow.cs 12-16; SceneBuilder.cs 630-643	Spawn noktaları, takip hedefleri, altın dizileri ve survivor referanslarının yönetimi.
NavMeshAgent	SceneBuilder.cs 450-456, 499-505; SurvivorAgent.cs 62, 122; MonsterAgent.cs 47, 97-103	Karakter ve canavarın navmesh üzerinde yön bulması ve kontrollü hareket etmesi.
Animation / Animator	SceneBuilder.cs 270-335, 443-445, 492-494; SurvivorAgent.cs 353-360; MonsterAgent.cs 206-208, 269	Yürüme, koşma ve saldırı animasyonlarının parametre tabanlı kontrolü.

4. Kavramların Ayrıntılı İncelemesi

4.1 Prefab Kullanımı

Projede prefab mantığı hem editör tarafında hem de oyun anında aktif şekilde kullanılmıştır.

- NocturneVillageSceneBuilder.cs içinde VillageHouse, StreetLamp, AutumnTree, ForestRock, GoldPickup, ShotTracer, RemySurvivor ve VampireMonster prefabları üretilmektedir.

- SceneBuilder tarafındaki 427-528 satır aralığında karakter prefablarına Animator, NavMeshAgent, collider ve gameplay scriptleri otomatik bağlanmaktadır.
- NocturneScenarioDirector.cs 199-238 satırlarında bu prefablar Instantiate edilerek sahneye canlı oyun nesneleri olarak yerleştirilir.

4.2 Vector3 ve Quaternion Kullanımı

Mekânsal kontrolün tamamı Vector3 ve Quaternion hesaplarıyla kurulmuştur.

- SurvivorAgent.cs 100-126 satırları hareket yönünü Vector3 ile oluşturur; LookRotation ve Slerp ile karakterin bakış yönünü hesaplar.
- SurvivorAgent.cs 303-309 satırlarında mermi çıkış noktası fireOffset ve Vector3.up kullanılarak hesaplanır.
- CameraRigFollow.cs 58-63 satırları kameranın hedef merkezine göre pozisyon ve rotasyonunu belirler.
- SceneBuilder.cs 602-643 ve 736-911 aralıkları sahnedeki yol, altın, ev ve orman elemanlarının koordinatlarını Vector3 ile tanımlar.

4.3 Partikül Efektleri

Projede atmosfer ve geri bildirim amacıyla çok sayıda partikül efekti kullanılmıştır.

- SceneBuilder.cs 358-372 aralığında BloodBurst, SpawnMist, ImpactBurst, MuzzleFlash, GoldBurst, AutumnDrift ve GroundFog efekt prefabları hazırlanır.
- GoldCollectible.cs 52-54 satırlarında altın toplandığında pickup efekti üretilir.
- MonsterAgent.cs 220 ve 255 satırlarında saldırı ve ölüm anlarında efektli geri bildirim verilir.
- Bu kullanım, projenin görsel atmosferini güçlendirirken oyuncu etkileşimlerini daha anlaşılır hale getirir.

4.4 Instantiate Kullanımı

Çalışma anında nesne üretimi için Instantiate ana yapı taşlarından biridir.

- NocturneScenarioDirector.cs 199-238 satırlarında survivor, monster ve gold nesneleri spawn edilir.
- SurvivorAgent.cs 309-339 satırlarında muzzle flash, impact burst ve shot tracer anlık üretilir.
- EscapeZone.cs 35 ve GoldCollectible.cs 54 satırlarında etkileşim anında sahneye efekt eklenir.
- Böylece önceden sahneye yerleştirilmemiş nesneler ihtiyaç anında kontrollü olarak oluşturulur.

4.5 RayCast Kullanımı

Projede hem saldırı hem de algılama sisteminde ışın testleri kullanılmıştır.

- SurvivorAgent.cs 312-326 satırları oyuncunun ateş ettiği doğrultuda Physics.RaycastAll çağırır ve ilk uygun hedefi tespit eder.
- MonsterAgent.cs 178-182 satırlarında canavarın hedefi gerçekten görüp görmediği Physics.Raycast ile kontrol edilir.
- Bu sayede hem mermi sistemi hem de yapay zekâ görüşü fizik katmanına bağlı olarak çalışır.

4.6 FixedUpdate, LateUpdate ve DeltaTime Kullanımı

Projede hareket, kamera ve zamana bağlı görsel akış farklı update döngülerine dağıtılmıştır.

- SurvivorAgent.cs 93-126 satırlarında karakter hareketi FixedUpdate içinde Time.fixedDeltaTime ile ilerletilir; bu seçim fizik kararlılığı sağlar.
- CameraRigFollow.cs 19-63 satırlarında kamera, LateUpdate içinde Time.deltaTime ile hedefleri takip eder; böylece karakter hareketinden sonra düzgün hizalanır.
- GoldCollectible.cs 26-27 ve ShotTracer.cs 20-37 aralığında objelerin dönme, salınım ve sönme davranışları delta time ile akıcı hale getirilir.

4.7 Trigger Kullanımı

Bu projede Collision yerine Trigger tabanlı etkileşim tercih edilmiştir.

- GoldCollectible.cs 17-21 satırlarında SphereCollider trigger moduna alınır ve 37-58 satırlarında altın toplama mantığı OnTriggerEnter ile yürütülür.
- EscapeZone.cs 17-35 satırları kaçış alanının collider tetikleyicisi olarak çalışmasını sağlar.
- Sebep şudur: altın ve çıkış kapısı gibi alanlarda fiziksel itme değil, bölgesel algılama gereklidir; Trigger bu yapı için daha doğrudur.

4.8 Arrays Kullanımı

Diziler, projenin çok oyunculu ve çok nesneli yapısını düzenlemek için yoğun biçimde kullanılmıştır.

- NocturneScenarioDirector.cs başında survivorPrefabs, survivorSpawnPoints, goldSpawnPoints ve spawnedSurvivors dizileri tanımlanır.
- Director içindeki 193-243 satır aralığı bu dizileri döngü ile gezerek tüm oyun kurulumunu otomatik yapar.
- CameraRigFollow.cs 12-16 satırlarında kamera takip hedefleri Transform[] dizisi üzerinden alınır.
- SceneBuilder.cs 630-643 aralığında altın pozisyonları bir Vector3[] dizi halinde tanımlanarak toplu üretim yapılır.

4.9 NavigationMesh ve NavMeshAgent Kullanımı

Yol bulma ve takip sistemi Navigation Mesh tabanlıdır.

- SceneBuilder.cs 450-456 ve 499-505 satırlarında survivor ve monster prefablarına NavMeshAgent bileşeni eklenir ve özellikleri ayarlanır.
- SurvivorAgent.cs 122 satırı agent.Move ile kontrollü oyuncu hareketini yürütür.
- MonsterAgent.cs 97-103 satırları SetDestination kullanarak canavarı hedefin son görüldüğü konuma taşır.
- Bu sistem, kasaba ve orman sahnesinde yapay zekânın engelleri doğal biçimde dolaşmasını sağlar.

4.10 Animation ve Animator Kullanımı

Animasyon sistemi hem karakterlerde hem de canavarda parameter-based animator mantığıyla kurulmuştur.

- SceneBuilder.cs 270-335 satırlarında survivor ve monster için Animator Controller oluşturulup Idle, Run, Walk ve Attack state'leri tanımlanır.
- SceneBuilder.cs 443-445 ve 492-494 satırları bu controller'ları prefabların Animator bileşenine bağlar.
- SurvivorAgent.cs 353-360 satırlarında Speed parametresi güncellenir; MonsterAgent.cs 206-208 satırlarında Attack trigger'ı ateşlenir.
- Böylece hareket hızına bağlı animasyon geçişleri ve saldırı animasyonu otomatik olarak oynatılır.

5. Sonuç

Nocturne Blood Run projesi, ders kapsamında beklenen Unity Scripting-Visualization Basics başlıklarının tamamını işlevsel bir oynanış akışı içinde birleştirmektedir. Prefab yapısı sayesinde sahne modüler hale getirilmiş; Vector3, Quaternion, RayCast, Trigger, arrays ve NavMeshAgent gibi bileşenler doğrudan oynanış mekaniklerine bağlanmıştır.

Ayrıca Animator tabanlı animasyonlar, partikül efektleri ve otomatik sahne üretimi ile proje yalnızca teknik açıdan değil, görsel anlatım açısından da güçlendirilmiştir. Bu nedenle çalışma; hem oyun mekaniği tasarımı hem de Unity bileşenlerinin uygulamalı kullanımı bakımından bütünlüklü bir örnek proje niteliğindedir.